

방금 그 화면을 만든 사람은 3D 기술을 모릅니다.

무엇을 배워서 만들 수 있었는지 — 3교시 끝에 공개합니다.

■ 오늘의 지도

네 시간, 하나의 이야기

1교시 · 언어

"에이전트"라는 말, 차분히 풀어보기

Agent · Connector · Plugin · Skill

2교시 · 엔진

일을 맡길 모델은 어떻게 고르는가

프론티어 모델 선별

3교시 · 작업대

그 모델을 어떤 도구에서 부리는가

Claude Desktop vs Claude Code

4교시 · 손

직접 만들어보고 퇴근한다

내 업무 Skill 1개 + Connector 연결

1

언어

요즘 어디서나 들리는 "에이전트"라는 말 —
그 뜻을 차분히 풀어보는 것에서 시작합니다.

■ 2022년 겨울

"이게 무슨 AI냐"

ChatGPT가 처음 나왔을 때를 기억하십니까.

말장난이나 한다고, 계산도 틀린다고 웃음거리가 됐습니다.

그 물건을 만드는 데 **수조 원**이 들었습니다.

■ 오늘의 질문

이 어려운 일을 —
어디까지 직접 만들고,
어디서부터 가져다 쓰면 될까요?

답을 정해두지 않겠습니다. 4시간 동안 같이 들고 갑니다.

■ 에이전트의 진짜 정의

목표를 향해, 스스로 반복하는 것



신입사원에게 "경쟁사 조사해 와"라고 하면 — 검색하고, 보고, 부족하면 다시 찾습니다.

이 루프가 에이전트입니다.

ReAct, Plan & Execute 같은 기법 이름은 몰라도 됩니다 — 즉흥형·계획형·분업형 일하기 방식 정도면 충분합니다.

■ 좋은 소식

에이전트는 이미 만들어져 있습니다.

Claude Desktop, Claude Code —

오늘 노트북에 설치할 수 있는 완성품입니다.

■ 모델만큼 중요한 것

에이전트의 품질 = 모델 × 설계

설계란 — 지시를 얼마나 넣고 얼마나 **덜어낼지**,
관찰을 몇 번 시킬지, 얼마나 믿고 맡길지.

Claude Code에 다른 회사 모델을 끼우면 원래 성능이 안 나옵니다.
같은 엔진도 차체가 다르면, 그 차가 아닙니다.

이 이야기는 2교시에서 "하네스"라는 이름으로 다시 만납니다.

■ "1인 1에이전트"를 열어보면

빌더 안에서 우리가 하는 일은, 이미 Skill을 쓰는 일입니다

우리가 하는 일

지시문을 쓰고, 도구를 고른다

절차와 기준을 글로 적는 일 — 오늘 배울 Skill과 정확히 같은 일입니다

플랫폼이 하는 일

관찰 · 재시도 · 오류 대처 — 루프 전체

이 부분은 빌려 쓰는 것이라, 플랫폼을 떠나면 함께 반납됩니다

일하는 에이전트는 곱셈의 결과입니다

내가 쓴 문서

Skill · 역할 정의
절차 · 판단 기준 · 도구 목록

내 것 — 이식 · 상속

×

빌린 실행력

루프 · 재시도 · 위임
(플랫폼의 하네스)

구독 — 계약 종료 시 반납

=

일하는 에이전트

목표를 향해 관찰하고
재시도하는 실행 인스턴스

둘이 결합된 순간에만 존재

참

"그것은 에이전트다" — 결과물을 가리키는 말

절반의 참

"내가 에이전트를 만들었다" — 곱셈의 왼쪽
항만 말한 문장

■ 그래서 "1인 1에이전트"는

한 문장이 아니라, 두 문장의 겹침입니다

"1인 1에이전트"

사용의 독해

참

"1인당 에이전트 하나씩을 부린다"

1인 1PC와 같은 문장. 이미 실현 가능하고, 바람직한 미래입니다.

소유의 독해

확인 필요

"1인당 에이전트 하나씩을 만들어 소유한다"

계약이 끝나면 함께 사라지는 것을 소유라 부르긴 어렵습니다. 남은 것은 문서입니다.

같은 말이 두 뜻을 오갈 때 — 왼쪽을 보고, 오른쪽에 서명하게 되기 쉽습니다

■ 틀린 말일까요?

"내 사무실"은 틀린 말이 아닙니다

대화에서

"내 사무실로 오세요"

임대 사무실이어도 다들 이렇게 말합니다. 자연스러운 말입니다.

등기부등본에서

"내 소유의 사무실"

문서에 적는 순간 이야기가 달라집니다. 건물주는 따로 있습니다.

"1인 1에이전트"도 같습니다 — 구호로는 자연스럽지만,
예산·자산·인수인계 계획서에 적히는 순간 소유의 문장이 됩니다.

■ 구호 완성하기

1인 1에이전트 (사용)

1인 N Skill (소유)

이 구호는 틀린 게 아니라 절반만 적혀 있었습니다.

나머지 절반 — 소유할 자산 — 을 오늘 채웁니다.

■ 금융사라면 미리 알아둘 것

시연과 도입 사이에는 간격이 있습니다

- 시연은 가장 좋은 모델 + 거기에 맞춘 설계로 이뤄집니다.
- 실제 도입되는 사내 모델이 같은 수준이라는 보장은 없습니다.
- 그런데 내부는 블랙박스라, 도입 후에야 알게 됩니다.

누구의 잘못도 아닌 구조의 문제입니다 — 그래서 질문을 준비해 가는 겁니다.

■ 가져가실 것

벤더에게 드릴 세 가지 질문

질문 ①

시연에 쓴 모델과 우리가 도입
할 모델이 같습니까?

질문 ②

우리 모델 기준으로 최적화됐
다는 근거는 무엇입니까?

질문 ③

이 플랫폼을 떠나면, 우리가 만
든 것 중 무엇이 남습니까?

세 번째 질문의 답이 — 오늘 강의의 전부입니다.

플랫폼을 떠나도 남는 것, 어디로든 이식 가능한 문서 — Skill

내 업무 노하우를 AI가 읽을 수 있는 표준업무절차서로 쓴 것.
파일이니까 내 것이고, 옮길 수 있고, 물려줄 수 있습니다.

■ Skill의 실체

특별한 기술이 아니라는 것, 그 자체가 설계입니다

실체는 문서 파일 하나. 맨 위에 "언제 꺼내 읽을지" 한 줄,
그 아래에 업무 절차.

규정집을 통째로 외우는 직원은 없습니다 —

필요할 때 해당 페이지를 펴는 직원이 유능한 겁니다.

기술 용어로는 progressive disclosure — AI의 주의력(토큰)을 아끼는 구조입니다.

■ 역사적 검증

엇갈린 반응에서 표준까지, 5개월

2025.10.16

Anthropic, Skill 발표. 반응은 갈렸다 — "그냥 텍스트 아니냐" vs "MCP보다 큰 물건"(사이먼 윌리슨)

2025.12.18

오픈 표준으로 공개

+48시간

Microsoft · OpenAI가 채택 — 경쟁사들이 서로의 형식을 그대로 읽기 시작

2026.03

Google 포함 32개 이상의 도구가 같은 파일을 읽는다 — 사실상의 산업 표준

■ 왜 "여러분의" 자산인가

빌더 안의 설정 vs 내 문서

	노코드 빌더 안의 설정	문서로 관리하는 Skill
소유	플랫폼 계약과 함께 종료	내 파일 — 영구 소유
이동	플랫폼 안에서만	경쟁사 AI들도 같은 파일을 읽음
인수인계	계정 이관 문제	파일 전달 = 노하우 전달

1년 전엔 "한 회사에서만 되는 기능 아니냐"고 할 수 있었습니다. 지금은 아닙니다.

오늘 나오는 말은 네 개뿐입니다

용어	한 줄 정의	회사 비유	오늘
Skill	AI에게 주는 재사용 가능한 업무 지침서 (문서)	표준업무절차서	직접 만든다
Connector	AI가 외부 시스템에 접근하는 표준 연결 통로	시스템 권한 있는 창구	직접 연결한다
Plugin	Skill·설정·연결을 묶어 배포하는 패키지	팀 표준 업무 키트	개념만
Agent	위 재료로 루프를 도는 실행 주체	일 위임받은 주니어	이미 준비되어 있다

Connector는 MCP라는 기술 규격의 소비자용 이름입니다 — 외우실 필요 없습니다, 유인물에 있습니다.

■ LIVE DEMO

에이전트가 루프를 도는 모습을 지금 보여드리겠습니다

과제: 보도자료 3건을 읽고 경영진용 1페이지 비교 메모 작성

화면에서 중계합니다 — "지금 관찰했죠. 부족하다고 판단했죠. 다시 행동하죠."

■ 1교시 요약

"에이전트는 이미 준비되어 있다.
우리가 만들 것은,
에이전트에게 줄 일하는 법 — Skill이다."

■ 쉬는 시간 · 10분

10:00

다음 · 2교시 엔진 — 일을 맡길 모델은 어떻게 고르는가

R 키 — 카운트다운 다시 시작



2

엔진

벤치마크 표 대신 — 내 업무 문제 3개로
모델을 고르는 절차를 가져갑니다.

■ 벤치마크의 함정

수능 만점자가 우리 팀 보고서를 제일 잘 쓸까요?

모델 발표마다 점수표가 나옵니다.

그런데 그 시험 문제는 여러분의 업무가 아닙니다.

벤치마크는 수능 점수 — 필요한 건 우리 팀 실무 면접입니다.

■ 보이지 않는 절반

하네스 — 모델을 일에 매는 장비

하네스(harness)는 원래 말에 채우는 마구(馬具)입니다.

아무리 좋은 말도 마구 없이는 수레를 못 끕니다.

AI에서 하네스 = 모델을 실제 업무에 매어주는 실행 환경 일체.

여섯 가지 보이지 않는 일

① 업무 지침

여러분 글 앞에 이미 들어가 있는 수천 단어의 기본 지시

② 도구 상자

파일 · 검색 · 실행 — 무엇을 쥐어줄지

③ 권한 체계

뭘 그냥 하게 하고, 뭘 물어보게 할지

④ 기억 관리

대화가 길어지면 중간 정리를 해주는 서기

⑤ 반복 제어

몇 번 재시도, 언제 멈추고 보고할지

⑥ 분업

큰 일을 쪼개 보조 일꾼에게 위임

세 개의 층 — 우리가 만드는 층은 하나

모델

지능 그 자체

제조사

하네스

모델을 업무에 매는 실행 환경

제조사

Skill

내 업무의 절차와 기준

여러분 (4교시)

"어느 모델이 최고냐"는 반쪽 질문 —

"어느 묶음(모델+하네스)이 내 업무에 맞냐"가 온전한 질문입니다.

스펙이 아니라 성격으로 소개합니다

축 ①

깊이 생각하는가, 빠르게 처리하는가

축 ②

대화 상대인가, 일을 통째로 맡기는 상대인가

깊게 생각하는 시니어 · 균형 잡힌 기본값 · 빠르고 싼 대량 처리 담당 —
사람 소개하듯 보시면 됩니다.

비용 감각: 대량 분류 작업을 최상급 모델에 시키는 건, 임원에게 우편물 분류를 시키는 것.

성격은 숫자에서도 보입니다

모델 [생각 강도]	성공률	평균 비용	스텝 수
gpt-5.6-sol [max]	73%	\$8.39	61
claude-fable-5 [max]	70%	\$21.63	88
gpt-5.6-terra [max]	70%	\$4.95	76
claude-opus-4.8 [max]	59%	\$13.22	120
claude-sonnet-5 [max]	54%	\$26.40	268
gemini-3.1-pro [high]	12%	\$9.48	81

실제 저장소 91곳 · 신규 과제 113개 · 전 모델 동일한 표준 하네스로 측정 (deepswe.datacurve.ai, 2026-07 확인)

■ 표 읽는 법

같은 70%, 전혀 다른 성격

심사숙고형

claude-fable-5

119k 토큰 · 88스텝 · \$21.63 — 깊게 파고들어 도달

속전속결형

gpt-5.6-terra

72k 토큰 · 76스텝 · \$4.95 — 효율적으로 도달

점수 열만 보지 말고 **비용·스텝 열**을 보십시오.

"누가 낫냐"가 아니라 "누가 우리 일에 맞냐"가 중요합니다.

12%짜리 모델도 나쁜 모델이 아닐 수 있습니다 — 하네스와의 궁합, 측정 조건이 달랐습니다. 표 하나로 결론 내리지 않습니다.

■ [max]가 뭔가요

생각 강도 다이얼

요즘 모델에는 "얼마나 오래 생각할지" 다이얼이 있습니다.

표의 [max], [high]가 그 설정값입니다.

다이얼 최대 = 항상 최선이 아닙니다. 올려도 그대로거나, 오히려 내려가기도 합니다.

모든 결재 서류를 이사회 안건처럼 검토하면 회사가 멈춥니다 — 과제 난이도에 맞는 강도가 정답입니다.

■ LIVE DEMO

리더보드를 직접 읽어봅니다

deepswe.datacurve.ai — 정렬 기준을 점수에서 비용으로 바꿔보면

"점수 1등"과 "비용 대비 1등"은 **다른 모델**입니다.

벤치마크를 버리라는 게 아닙니다 — 점수 열만 보지 말고 성격 열을 읽자는 겁니다.

■ 가져가실 절차

내 업무 3문제 테스트법

- 자주 하는 과제 3개를 고른다 — 요약 1 · 판단 1 · 작성 1
- 모델을 보기 전에, 각 과제의 "좋은 결과의 기준"을 먼저 적는다
- 후보 모델들에 동일하게 시키고, 기준으로 채점한다
- 점수 + 속도 + 비용으로 결정하고 — 분기마다 재평가한다

모델은 계속 바뀝니다. 한 번의 선택이 아니라 절차를 가져가십시오.

■ 확장

레버리지가 개인에게 왔다

컴퓨터 한 대 + 로컬 모델 + 프론티어 모델의 감독
= 개인이 굴리는 콘텐츠 생산 라인.

2인 개발팀의 게임이 수백억 원대 수익을 내는 시대입니다.

강의 전 수치 검증

그 레버리지의 손잡이가 — 오늘 배우는 Skill이라는 이야기입니다.

■ 2교시 요약

"모델은 점수표가 아니
라 내 업무 3문제로 고른다.
그리고 언제나 **하네스와의 묶음**으로 고른
다."

■ 쉬는 시간 · 10분

10:00

다음 · 3교시 작업대 — Claude Desktop vs Claude Code

R 키 — 카운트다운 다시 시작

3

작업대

월요일 아침, 두 도구 중 무엇을 열지
3초 안에 판단하게 됩니다.

■ 본질적 차이

회의실에서 같이 일하기 vs 자리 내주고 위임하기

Claude Desktop

대화 상대

내가 자료를 가져가 보여주며 함께 일한다 — 화면 중심, 왕복 중심

Claude Code

내 컴퓨터에 앉힌 동료

폴더를 통째로 주고 "이 안에서 일해" — 파일 중심, 결과물 중심

이름에 Code가 있지만 코드 전용이 아닙니다 — 문서 40개 일괄 정리 같은 일에 강합니다.

■ 3 초 판별법

결과물이 대화인가, 파일들인가

상황	도구	이유
아이디어 탐색 · 문서 1~3개 요약	Desktop	왕복 대화가 빠르다
보고서 20건 → 비교표 1개	Code	폴더 단위 위임
매주 반복하는 정형 산출물	Code + Skill	절차의 자산화
결과를 화면에서 보며 다듬기	Desktop	즉시 피드백

Code도 자연어로 씁니다. 명령어 암기 불필요 — 권한을 물어오면 확인하고 승인하는 습관이면 충분합니다.

■ LIVE DEMO

같은 과제, 두 도구

과제: 샘플 문서 묶음으로 비교표 만들기

Desktop — 파일을 끌어 넣고 대화로. 즉각적입니다. 그런데 문서가 30개라면?

Code — 폴더를 주고 "전부 읽고 비교표 파일로 저장해줘".

파일이 생겨나는 순간을 보십시오.

■ 떡밥 회수

오프닝의 그 화면, 이렇게 만들었습니다

Before / After — 그리고 실제 사용한 프롬프트 전부 공개

구현은 AI의 몫, 연출은 나의 몫

"멋있게 만들어줘"

아쉬운 결과 — AI도 무엇이 멋있는지는 모릅니다

"카메라가 아래에서 위로 돌며,
스크롤에 따라 장면이 전환되고..."

성공 — 장면 · 연출 · 카메라 모션 · 스크롤 이벤트, 감독의 언어

■ 일반화

여러분의 "연출 언어"는 여러분 업무 안에 이미 있습니다

보험 실무자가 배울 것은 프로그래밍이 아니라 —

좋은 심사보고서의 구조 · 좋은 비교표의 기준 · 좋은 요약의 조건.

그것을 문서로 쓰면, 그게 Skill입니다. 다음 시간에 직접 만듭니다.

■ 3교시 요약

"대화면 Desktop, 파일 작업이면 Code.
배워야 할 것은 구현 기술이 아니라
내 업무의 **연출 언어**다."

■ 쉬는 시간 · 10분

10:00

다음 · 4교시 손 — 직접 만들어보는 시간 (노트북을 준비해주세요)

R 키 — 카운트다운 다시 시작

4

손

전원이 자기 업무 Skill 1개를 만들고,
Connector 1개를 연결해본 상태로 퇴근합니다.

■ 실습 ① · 25분

내 업무 Skill 만들기

- 최근 한 달간 2번 이상 반복한 업무 하나를 고른다
- 메모장에 3가지만 적는다 — ① 언제 하는 일 ② 일하는 순서 ③ 결과물의 형식
- Claude에게 붙여넣고 요청한다 (다음 장의 문장 그대로)
- 질문에 답하며 완성 → 등록 → 샘플로 실행
- 아쉬운 점 1가지를 지침에 추가한다 — Skill은 한 번에 완성되지 않고 기릅니다

■ 복사해서 쓰세요

요청 문장

"이걸 재사용 가능한 Skill(업무 지침서)로 만들어줘.
빠진 정보가 있으면 질문 5개까지만 해줘."

성공 기준은 화려함이 아닙니다 — 내 지침대로 결과 형식이 바뀌는 것을 눈으로 확인하는 것.

■ 실습 ② · 20분

Connector — AI에게 시스템 출입증 주기

- 설정 → Connectors → Gmail 연결 → 구글 로그인 승인
- 과제: "지난 2주 메일 중 아직 회신 안 한 것 같은 메일을 찾아 3줄씩 요약해줘"
- 그리고 반드시 — 연결 해제하는 법, 권한 확인하는 법

AI는 내 권한 이상은 절대 못 봅니다. 금융사에서는 붙이는 법보다 끊는 법을 아는 게 신뢰를 삽니다.

■ 강사 시연

공식 커넥터가 없는 시스템은? 창구를 직접 만듭니다

시연: "맥 비서" — 이 컴퓨터에서만 도는 창구(로컬 MCP)

상품 DB를 조회하고 → 결론을 소리 내어 읽고 → 완료 알림을 띄우고 → 후속 일정을 등록합니다.

■ 특별 코너 · 10분

실제 설계도 하나를, 오늘 배운 눈으로 같이 봅니다

신사업추진파트 — "Agent 활용 신사업 발굴 방법론" 도식

- 역할별 에이전트 12개의 **역할 정의서** — 문서로 남는 자산, 오늘 말한 Skill
- 오케스트레이터와 루프 — 빌려 쓰는 층, 플랫폼이 가장 잘하는 부분
- 단계별 사람 게이트 4개 — **이 설계의 백미**, 판단과 책임은 사람에게

■ 오늘의 네 문장

가지고 가실 것

- 에이전트는 이미 준비되어 있다 — 내가 만들 것은 Skill이다
- 모델은 점수표가 아니라 내 업무 3문제로, 하네스와의 묶음으로 고른다
- 대화면 Desktop, 파일 작업이면 Code
- 배워야 할 것은 구현 기술이 아니라 내 업무의 연출 언어다

■ Next Action

다음 주에 Skill을 **딱 하나**만 더 만들어보세요.

그 문서 파일이, 어느 플랫폼을 쓰든 들고 다닐 여러분의 자산입니다.

오프닝의 그 화면을 만든 건 기술이 아니라 연출 언어였습니다.

여러분의 연출 언어는 — 여러분 업무 안에 이미 있습니다.